

FITNESS TRACKER

FastAPI · OOP · Pytest · Railway

fitnesstracker-production-dc35.up.railway.app

Diyor Akobiov - 100007584

77

Tests

9.91

Pylint

8

Classes

25+

Endpoints

What It Does



User Profiles

Create and manage profiles with fitness level, goal, and available equipment



Workout Generator

Personalised plans based on profile, equipment, and BMI category



Session Logging

Strength (sets $\times$ reps $\times$ weight) and cardio (duration + distance) exercises



Weight Tracking

Log body weight over time with BMI calculation and trend monitoring



Search & Filter

Search sessions by keyword, filter by user, browse exercise database



Persistent Storage

Data survives server restarts via JSON file — configurable via env variable

Project Structure

OOP Core

`workout_tracker.py`

All classes and business logic — the main focus

FastAPI Backend

`main.py`

25+ HTTP endpoints wiring OOP layer to the web

Frontend

`index.html`

Single-page UI served directly by FastAPI

Test Suite

`test_workout_tracker.py`

77 tests across 11 classes, zero real I/O

Entry Point

`run.py`

Starts uvicorn, reads PORT env var for Railway

Data Store

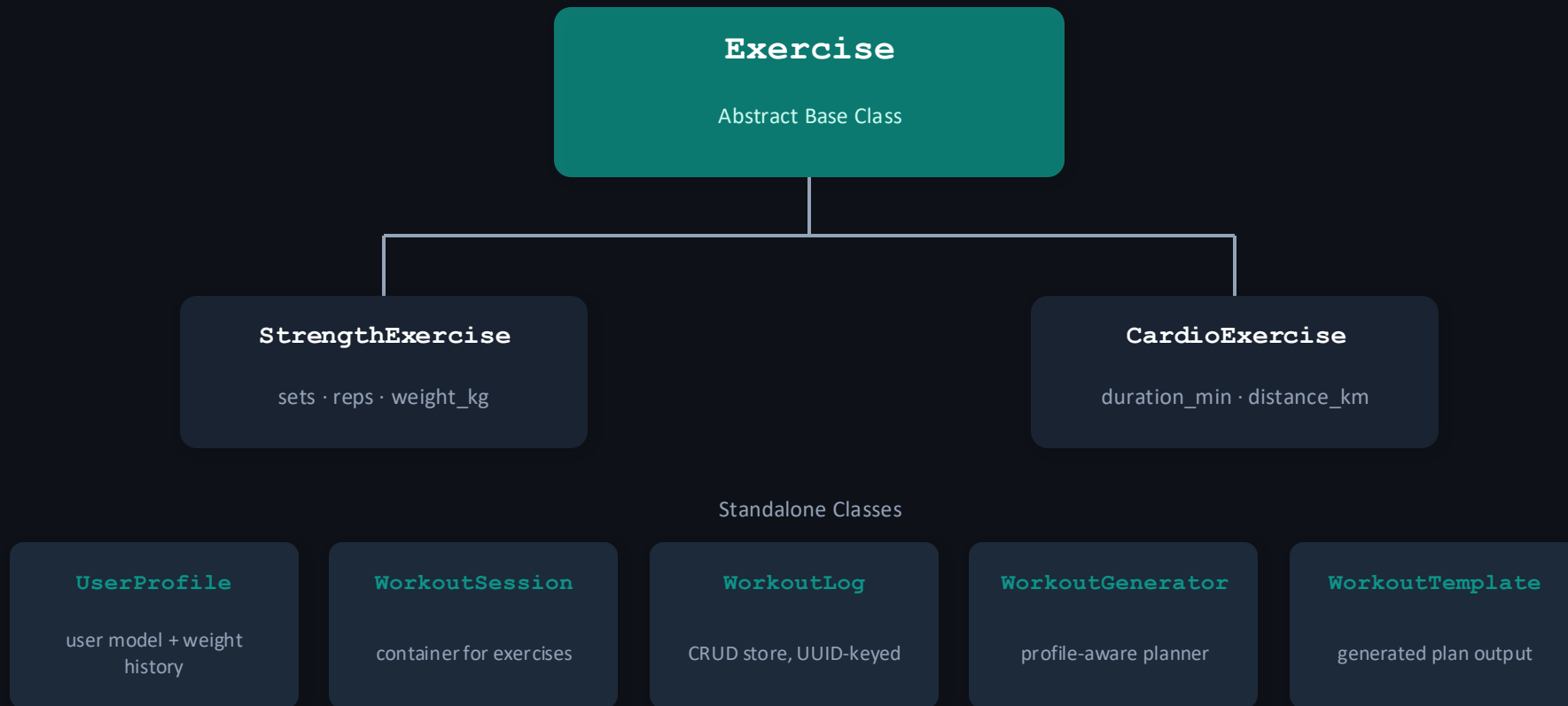
`workouts.json`

Auto-generated, git-ignored, configurable via DATA_FILE

Class Hierarchy

8 classes — each with a single, clear responsibility

Class Hierarchy



Encapsulation & Abstraction

Private attributes · Validated setters · ABC contract

Encapsulation & Abstraction

```
# Validated setter
@weight_kg.setter
def weight_kg(self, value):
    value = float(value)
    if value < 0:
        raise ValueError(...)
    self._weight_kg = round(value, 2)
```

```
# Read-only id
@property
def id(self): return self._id
# no setter
```

```
class Exercise(ABC):
    @abstractmethod
    def get_type(self) -> str: ...
    @abstractmethod
    def get_extra_fields(self) -> dict: ...
```

All attributes private

Every field stored as `_name`, `_sets`, `_weight_kg` etc. — never directly accessible from outside.

Validated setter on every field

Every setter raises `ValueError` on bad input: negative weight, empty name, wrong type.

Read-only id

@property getter, no setter — UUID is immutable after creation.

ABC contract enforced by Python

`TypeError` raised immediately if a subclass skips `get_type()` or `get_extra_fields()`.

Inheritance & Polymorphism

Subclasses extend, never re-implement · WorkoutSession stays subtype-agnostic

Inheritance & Polymorphism

```
class StrengthExercise(Exercise):
    def __init__(self, name, sets, reps,
                  weight_kg, ...):
        super().__init__(name, ...)
        self.sets = sets
        self.reps = reps
        self.weight_kg = weight_kg
```

```
# WorkoutSession — doesn't care about subtype
for ex in session.get_all_exercises():
    ex.get_type()           # "strength" or "cardio"
    ex.get_extra_fields()  # {sets, reps} or
    {duration}
    ex.to_dict()           # full serialised dict
```

`super().__init__()`

Both subclasses delegate shared setup upward — name, id, muscle_group, notes.

Own fields only

Strength adds sets/reps/weight_kg. Cardio adds duration_min/distance_km. Zero overlap.

Zero re-implementation

Validation, to_dict(), __repr__() — all inherited from Exercise. DRY enforced.

Open/closed in practice

Adding FlexibilityExercise requires zero changes to WorkoutSession or WorkoutLog.

FEATURE

Workout Generator

Profile-aware planning with goal modifiers and BMI adjustments

Workout Generator — How It Works

1

Filter exercises

Match equipment, fitness level, and workout type from the database

2

Apply goal modifiers

Adjust default sets and reps based on the user's fitness goal

3

BMI adjustment

Reorder exercises and optionally add extras based on BMI category

```
class ExerciseTemplate:
    """A blueprint exercise that can be included in a generated workout plan."""

    def __init__(
        # pylint: disable=too-many-arguments,too-many-positional-arguments
        self, name: str, muscle_group: str, exercise_type: str, equipment: str,
        workout_types: list[str], levels: list[str], default_sets: int = 3,
        default_reps: int = 10, default_duration_min: float = 20.0, description: str = "",
    ) -> None:
        """Initialise exercise template with suggested parameters."""
        self.name = name
        self.muscle_group = muscle_group
        self.exercise_type = exercise_type
        self.equipment = equipment
        self.workout_types = workout_types
        self.levels = levels
        self.default_sets = default_sets
        self.default_reps = default_reps
        self.default_duration_min = default_duration_min
        self.description = description

    def fits(self, workout_type: str, level: str, user_equipment: list[str]) -> bool:
        """Return True if this template suits the given workout type, level, and equipment."""
        return (
            workout_type in self.workout_types
            and level in self.levels
            and self.equipment in user_equipment
        )
```

BMI Adjustments

Higher BMI

Prioritises more cardio exercises

Underweight

Prioritises strength exercises

Either

Can add an extra exercise to the set

BACKEND

API Endpoints

25+ routes across profiles, sessions, exercises, stats, and metadata

API Endpoints

Metadata

GET /meta/workout-types

GET /meta/equipment

GET /meta/levels

GET /meta/goals

```
# -----  
# Generate  
# -----  
  
@app.post("/generate")  
def generate_workout(body: GenerateRequest):  
    profile = log.get_profile(body.profile_id)  
    if profile is None:  
        raise HTTPException(404, f"Profile '{body.profile_id}' not found.")  
    try:  
        template = generator.generate(profile=profile, workout_type=body.workout_type, focus_muscles=body.focus_muscles)  
    except ValueError as exc:  
        raise HTTPException(422, str(exc))  
    return template.to_dict()
```

```
# -----  
# Meta  
# -----  
  
@app.get("/meta/workout-types")  
def get_workout_types():  
    return [{"value": w.value, "label": w.value.replace("_", " ").title()} for w in WorkoutType]  
  
@app.get("/meta/equipment")  
def get_equipment():  
    return [{"value": e.value, "label": e.value.replace("_", " ").title()} for e in Equipment]  
  
@app.get("/meta/levels")  
def get_levels():  
    return [{"value": l.value, "label": l.value.title()} for l in FitnessLevel]  
  
@app.get("/meta/goals")  
def get_goals():  
    return [{"value": g.value, "label": g.value.replace("_", " ").title()} for g in FitnessGoal]  
  
@app.get("/meta/muscles")  
def get_muscles():  
    return [{"value": m.value, "label": m.value.replace("_", " ").title()} for m in MuscleGroup]
```

Generator & Stats

POST /generate

GET /stats

GET /exercises/database

GET / (serves frontend)

QUALITY

Testing & Code Quality

77 tests · Pylint 9.91/10 · Mypy clean pass

Test Suite — 77 Tests, 11 Classes

77

Total Tests

11

Test Classes

9.91

Pylint Score



Mypy Clean

Test Classes

●	TestStrengthExercise	Field validation, volume, serialisation
●	TestCardioExercise	Duration/distance, pace, serialisation
●	TestExerciseIdentity	UUID uniqueness, read-only id, equality
●	TestUserProfile	Age/level/goal/equipment/height/weight, BMI
●	TestWorkoutSession	Add/remove, total volume, cardio minutes
●	TestWorkoutLogSessions	CRUD — add, get, update, delete, search

Pylint 9.91/10

Only warnings: all false positives on data model constructors

R0913/R0917 `UserProfile.__init__`
Needs 8 args by design — legitimate data model

R0913/R0917 `WorkoutSession.__init__`
Same — multiple required fields

R0902 `WorkoutLog`
10 attributes appropriate for a CRUD store

DEPLOYMENT

Running Locally & Railway

python run.py · DATA_FILE env var · Persistent volume

Running Locally & Deploying to Railway

Local Setup

1

```
git clone  
https://github.com/Diyor7060/fitness-  
tracker.git
```

2

```
python -m venv venv  
venv\Scripts\activate
```

3

```
pip install -r requirements.txt
```

4

```
python run.py
```

Railway Deployment

1. Push to GitHub

Connect repo to
Railway

2. Set start command

```
python run.py
```

3. Add Volume

Mount path: /data

4. Set env variable

```
DATA_FILE =  
/data/workouts.json
```

5. Generate domain

Railway injects PORT
automatically

Tech Stack

Language

Python 3.11+

Backend

FastAPI

Server

Uvicorn

Frontend

HTML + CSS + Vanilla JS

Storage

JSON file

Testing

pytest with fixtures + tmp_path

Static Analysis

Pylint 9.91/10

Type Checking

Mypy — clean pass

Hosting

Railway

Fitness Tracker

OOP

ABC · Encapsulation · Inheritance · Polymorphism

Backend

FastAPI + Uvicorn · 25+ endpoints · JSON persistence

Quality

77 tests · Pylint 9.91 · Mypy clean · pytest fixtures

Live

fitnesstracker-production-dc35.up.railway.app